DISTRIBUTED SYSTEMS · GROUP PROJECT

Mini Distributed File Storage

Files that survive a crashed server: an HDFS-style storage system in Python and Flask



[Group members] · [Course and lecturer] · October 2026

Big data needs more than one server



Big data outgrows one disk

A single machine has fixed space, so large datasets have to be spread across many machines.



Machines fail

When the only server stops, every file on it becomes unreachable until it is repaired.

OUR QUESTION

Can a file still be downloaded, complete and unchanged, after a machine that stored it has stopped?

Three ideas from HDFS keep every file safe

1

Split

Every file is cut into 1 MB chunks, spread across three storage nodes.

2

Replicate

Every chunk is stored on 2 different nodes, so losing one node never loses data.

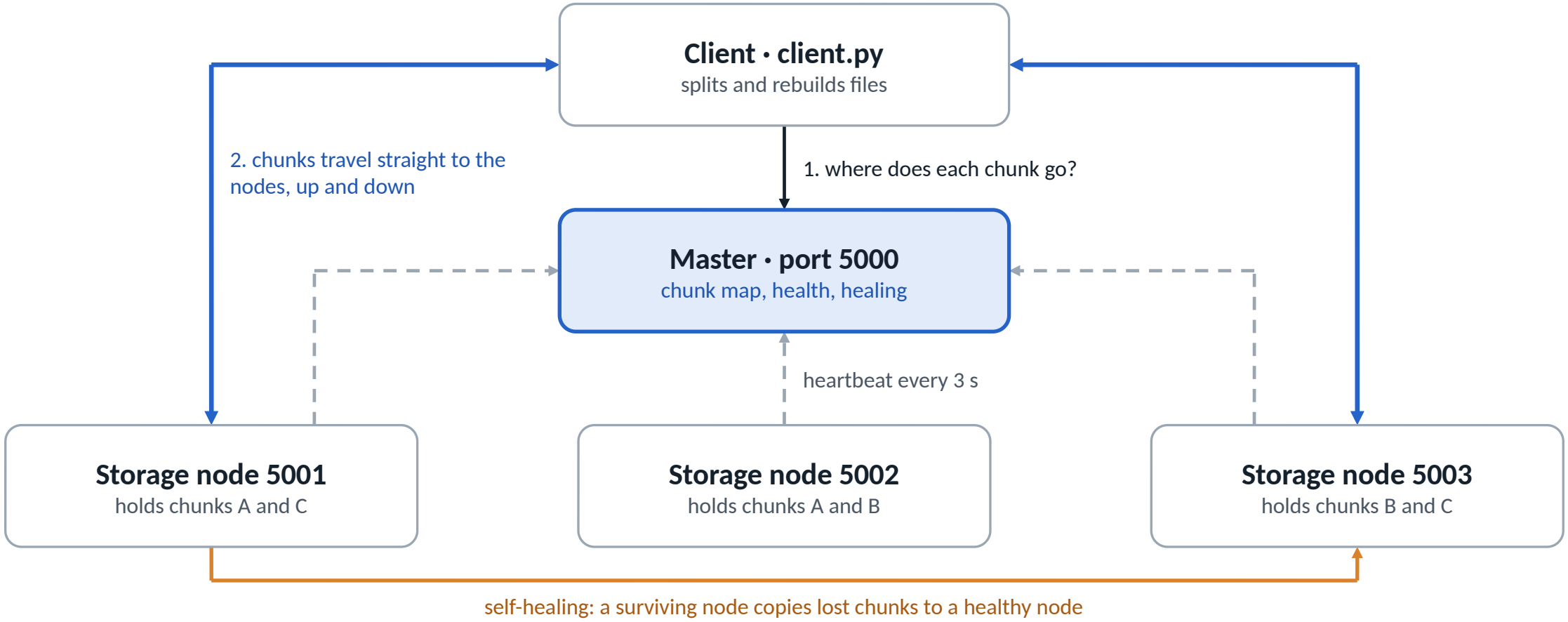
3

Detect and heal

Heartbeats every 3 s reveal a dead node within 10 s; its chunks are copied to a healthy node.

Integrity: SHA-256 checksums on every chunk and every file prove a download is unchanged.

The master holds the map; nodes hold the data



Reads never stop; healing takes 10 s



Measured in tests T3 and T4. The master waits for 10 s of silence before copying, so a node that is only briefly slow does not trigger needless copies.

Live demo: crash a node, get the file back

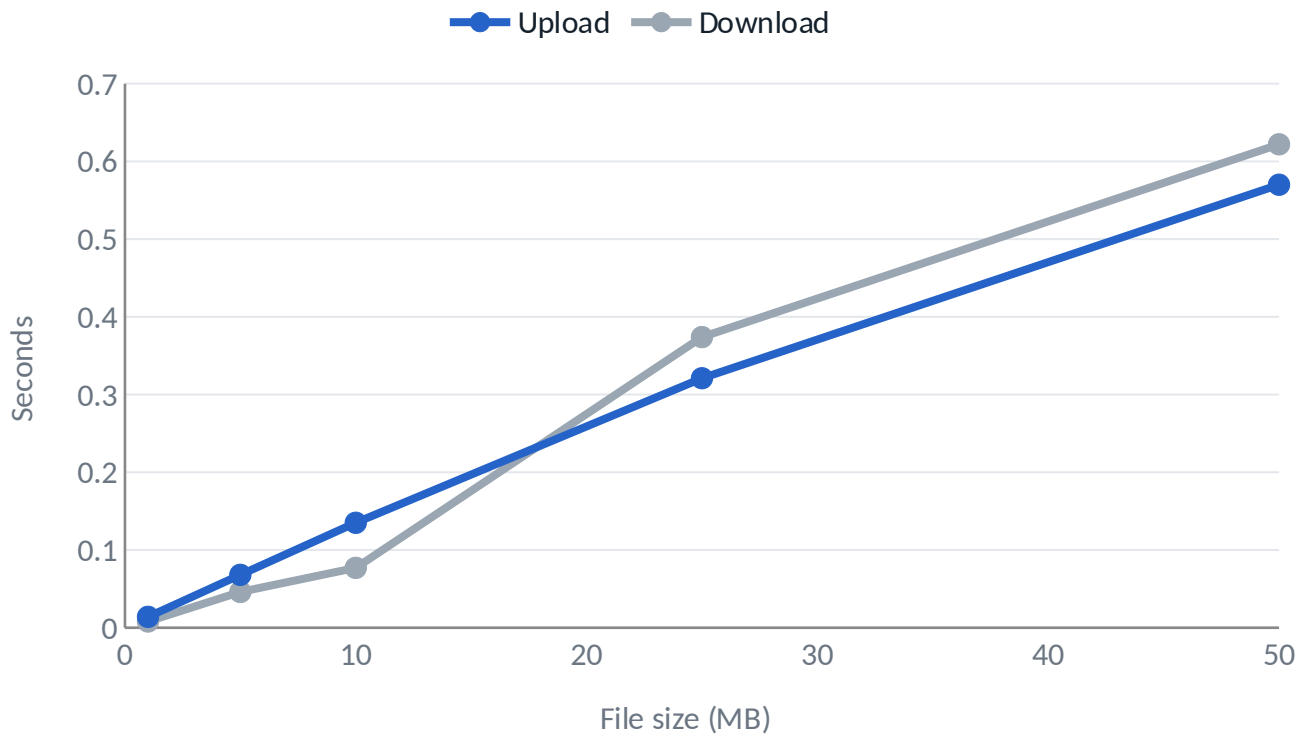
STEP	COMMAND	WHAT YOU SEE
1 Start the cluster	`python cluster.py --fresh`	Three nodes join the master
2 Upload a 3 MB file	`python client.py upload samples/reviews.csv`	3 chunks, each stored on 2 nodes
3 Show the chunk map	`python client.py map reviews.csv`	An X for every copy a node holds
4 Crash a node	`stop 5002`	Process killed, no warning
5 Download the file	`python client.py download reviews.csv`	Checksum OK; 1 chunk read from its backup copy
6 Wait 10 seconds	`(watch the master log)`	“node 5002 is DEAD”, then re-replication
7 Show the map again	`python client.py map reviews.csv`	Every chunk back to 2/2 live copies
8 Bring the node back	`start 5002`	It rejoins; stale copies are removed

All 7 tests passed, including key test T3

#	TEST	WHAT WE OBSERVED	RESULT
T1	Upload a 5 MB file	5 chunks, each on 2 different nodes	**Pass**
T2	Download, all nodes running	SHA-256 matches the original; 0.04 s	**Pass**
T3	Stop one node, then download	Complete in 0.05 s; checksum matches	**Pass**
T4	Keep it stopped for 10+ s	Marked dead; its 3 copies re-created	**Pass**
T5	Stop a second node	Downloads from node 5001 alone; checksum matches	**Pass**
T6	Upload 10 files, mixed sizes	30 / 30 / 30 chunk copies per node	**Pass**
E1	Restart both nodes (extra)	Both rejoin; 3 stale copies removed	**Pass**

Automated in run_tests.py: nodes are killed like real crashes, test files are random, and each node's disk is checked, not only the master's map.

Fast transfers, evenly balanced storage



0.57 s

to upload 50 MB, with every chunk written to 2 nodes

30/30/30

chunk copies on each node after 10 uploads (T6)

Time per file, median of 5 runs at each size from 1 to 50 MB, on one laptop

What this prototype does not do yet

Known limitations

- The master is a single point of failure; real HDFS adds a standby master
- All nodes share one laptop, so real network delays are untested
- Files can be replaced, not edited
- No accounts, access control or encryption

What we would build next

- A standby master with a copy of the metadata
- Storage nodes on separate machines
- Rack-aware placement of the two copies
- Background checksum scans of stored chunks
- A small Django dashboard of nodes and chunks

WHAT WE LEARNED

Split the data, keep two copies, listen for heartbeats, and storage survives a crash.

Every test passed: files came back complete and unchanged with one, and even two, of three nodes down.

Thank you. Questions?

Code, tests, report and slides: [GitHub repository link]